

Python을 이용한 파일 처리

04 패턴 이용하기

목차 04 패턴 이용하기

1. 패턴으로 파일 찾기
2. 간단한 정규 표현식

1. 패턴으로 파일 찾기

- 특수한 의미를 가지는 문자(와일드카드)로 여러 파일 이름을 지정하는 방법
- Windows 및 macOS의 명령줄에서도 사용 가능
 - 사용 환경에 따라 문법이 다를 수 있음
- Path에 사용 가능한 glob 패턴 관련 메서드
 - **match**: 경로가 glob 패턴과 일치하는지 확인
 - **glob**: 경로에서 glob 패턴을 이용하여 파일 산출
 - **rglob**: **glob**과 같으나 재귀적으로 실행



- *****
 - 0개 이상의 모든 문자와 일치
- **?**
 - 하나의 모든 문자와 일치
- **[characters]**
 - characters 중 하나 이상에 포함되는 하나의 문자와 일치
- **[a-z]**
 - a에서 z 중 하나 이상에 포함되는 하나의 문자와 일치



- **[!characters]**
 - characters에 포함되지 않는 하나의 문자와 일치
- **[!a-z]**
 - A에서 z 에 포함되지 않는 하나의 문자와 일치
- ******
 - 이 디렉터리와 모든 하위 디렉터리 일치



Path.glob을 사용한 용례 확인(*)

Python을 이용한 파일
처리

03 여러 파일 다루기

1. 패턴으로 파일 찾기



디렉터리 구조

```
from pathlib import Path

p = Path("dir")

for file in sorted(p.glob("*.txt")):
    print(file)
    # dir\textfile1.txt
    # dir\textfile2.txt
```

특정 확장자를 가진 파일을 추출할 때 사용

"*.txt" → 확장자가 .txt인 모든 파일 추출



Path.glob을 사용한 용례 확인(?)

Python을 이용한 파일
처리

03 여러 파일 다루기

1. 패턴으로 파일 찾기



디렉터리 구조

```
from pathlib import Path

p = Path("dir")

for file in sorted(p.glob("file?.txt")):
    print(file)
    # file1.txt
    # file2.txt
```

0-9까지의 번호로 이루어진 파일을 추출할 때 사용

"file?.txt" → file0.txt, file1.txt, ..., file9.txt 모두 추출



Path.glob을 사용한 용례 확인([])

Python을 이용한 파일
처리

03 여러 파일 다루기

1. 패턴으로 파일 찾기



디렉터리 구조

```
from pathlib import Path

p = Path("dir")

for file in sorted(p.glob("file[A-Z].txt")):
    print(file)
    # fileA.txt
    # fileB.txt
    # fileC.txt
```

특정 문자를 포함하는 파일을 추출할 때 사용

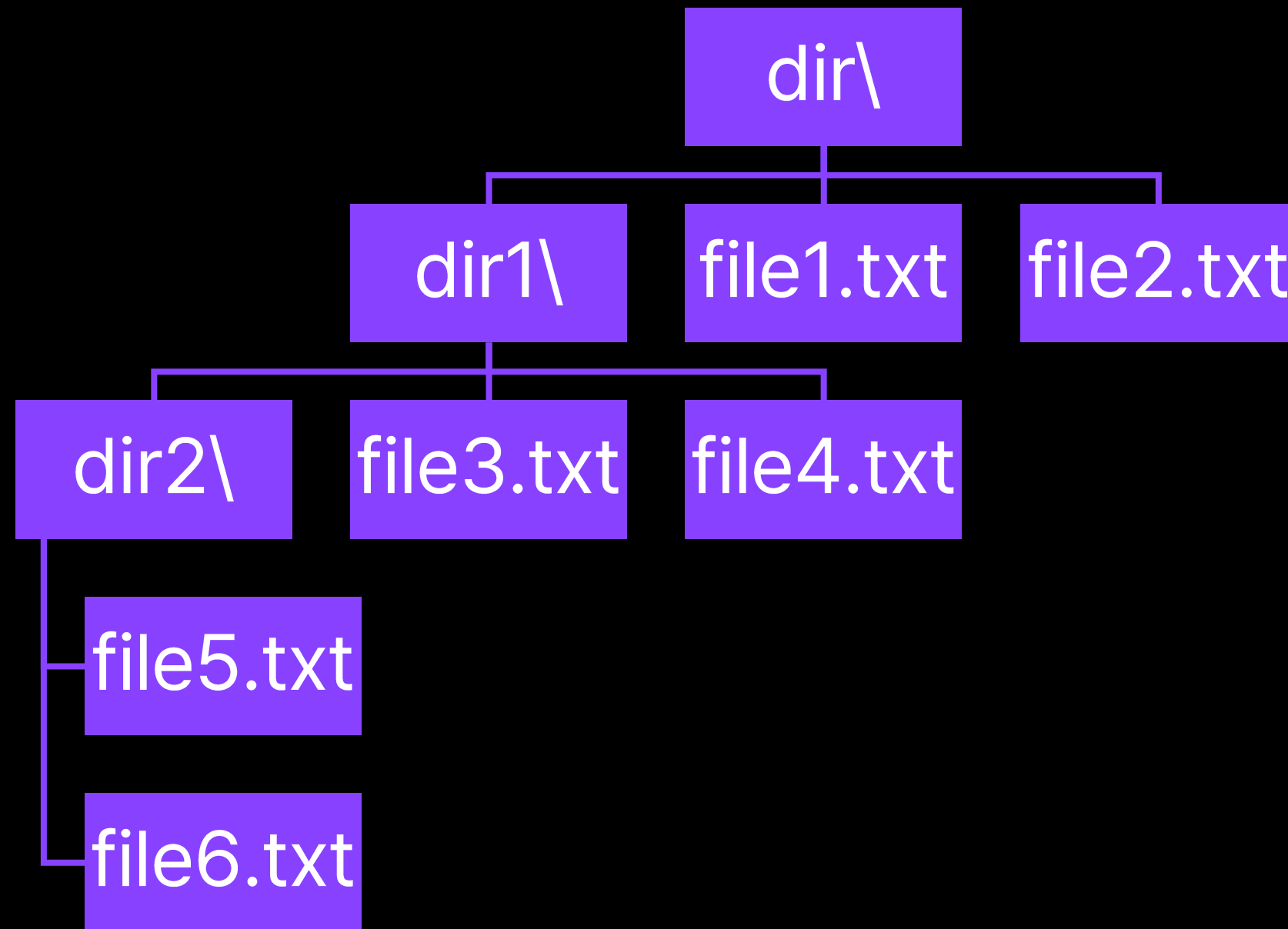
"file[A-Z].txt" → fileA.txt, fileC.txt, fileZ.txt
모두 추출

Path.glob을 사용한 용례 확인(**)

Python을 이용한 파일
처리

03 여러 파일 다루기

1. 패턴으로 파일 찾기



디렉터리 구조

```
from pathlib import Path

p = Path("dir")

for file in sorted(p.glob("**/*")):
    print(file)
# dir\dir1
# dir\dir1\dir2
# dir\dir1\dir2\file5.txt
# dir\dir1\dir2\file6.txt
# dir\dir1\file3.txt
# dir\dir1\file4.txt
# dir\file1.txt
# dir\file2.txt
```

디렉터리 순회시 사용

"**/*" → 현재 디렉터리에서 하위 디렉터리까지 모든 파일 선택



```
from pathlib import Path

p = Path("file1.txt")

print(p.match("file?.txt"))
# True
print(p.match("file?.csv"))
# False
```

- **match**
- Path 객체가 glob 패턴과 일치하는지 확인

```
from pathlib import Path

p = Path("dir")

for file in sorted(p.glob("*.txt")):
    print(file)
# dir\textfile1.txt
# dir\textfile2.txt
```

- **glob**
- Path 객체가 가리키는 디렉터리에
서 glob 패턴에 맞는 파일을 산출
하여 **제너레이터**로 반환
- 디렉터리가 아닌 경로에 대해서는
빈 제너레이터 반환

```
from pathlib import Path

p = Path("dir")

for file in sorted(p.rglob("*")):
    print(file)
# dir\dir1
# dir\dir1\dir2
# dir\dir1\dir2\file5.txt
# dir\dir1\dir2\file6.txt
# dir\dir1\file3.txt
# dir\dir1\file4.txt
# dir\file1.txt
# dir\file2.txt
```

- **rglob**
- **glob("**/*")**와 완벽히 동일



```
from pathlib import Path

p = Path("A.txt")

print(p.match("[a-z].TXT", case_sensitive=True))
# False
print(p.match("[a-z].TXT", case_sensitive=False))
# True
```

- Python 3.12부터 사용 가능
- `match`, `glob`, `rglob` 모두 가지고 있는 매개 변수
- `glob` 패턴의 대소문자 구별 여부를 결정
- 전달하지 않거나 `None`을 전달할 경우 운영 체제에 따라 동작이 다름
 - Windows: `False`를 전달한 것과 동일(구별하지 않음)
 - macOS: `True`를 전달한 것과 동일(구별함)
- 사용 가능한 경우에는 호환성을 위해서는 `None` 이외의 값을 전달하는 것이 좋음



```
from pathlib import Path

def rm_txt_from_dir(path):
    """path 및 하위 디렉터리에서 확장자가 .txt인 파일 모두 제거"""
    dir_path = Path(path)

    # 디렉터리에서 확장자가 .txt인 파일 모두 선택
    files = dir_path.rglob("*.txt", case_sensitive=True)

    # txt 파일 모두 제거
    for file in files:
        # 디렉터리가 아닌지 확인
        if file.is_file():
            file.unlink()
```

디렉터리 안에서 패턴에 맞는 특정 파일만 골라 삭제하거나 이동, 복사, 혹은 이름 변경 가능



glob 패턴을 이용한 파일 정리 사례

Python을 이용한 파일
처리

03 여러 파일 다루기

1. 패턴으로 파일 찾기

```
from pathlib import Path

def org_files_by_ext(path):
    """파일을 확장자에 맞게 정리"""
    dir_path = Path(path)

    # 디렉터리에서 확장자가 있는 파일 모두 선택
    files = dir_path.glob("*.*", case_sensitive=False)

    for file in files:
        # 실제 파일인지 확인
        if file.is_file():
            # 소문자 확장자를 사용하는 디렉터리 생성
            new_path = dir_path / (file.suffix[1:].lower() + "_file")
            new_path.mkdir(exist_ok=True)

            # 새 파일 위치 정의
            new_file_path = new_path / file.name

            # 파일 이동
            file.replace(new_file_path)
```

디렉터리 안에서 패턴에 맞는 특정 파일만 골라 삭제하거나 이동, 복사, 혹은 이름 변경 가능

2. 간단한 정규 표현식



```
import re

string = "궁금하신 사항이 있으시면 support@example.com으로 문의 부탁드립니다."
email = re.search(
    r"[a-z0-9!#$%&'*/+=?^_`{|}~-]+(?:\.[a-z0-9!#$%&'*/+=?^_`{|}~-]+)*@(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?",
    string,
)

if email:
    print(email.group())
    # support@example.com
```

- 특정한 문자열의 집합에 대한 표현에 사용되는 식
- glob 패턴보다 더 많은 집합을 함축적으로 표현 가능
- 와일드카드 대신 메타 문자라는 표현 사용
- 복잡한 식은 매우 난해함 → **항상 테스트 필요**
- <https://regex101.com/> 에서 작성한 정규 표현식을 테스트할 수 있음



. ^ \$ * + ? { } [] \ | ()

- glob 패턴의 와일드카드와 마찬가지로 기본적으로 정규표현식은 문자에 그대로 대응하나 몇 문자는 특수한 의미를 가짐 → 메타 문자
- glob 패턴은 와일드카드를 직접 표현할 수 있는 방법이 없음
- 정규 표현식의 경우 메타 문자 앞에 **'\'**를 붙이면 해당 문자 자체를 의미함
- 예: 정규 표현식 "test\\.\\\"는 문자열 "test.\\\"에 대응



	glob 패턴	정규 표현식
0개 이상의 모든 문자와 일치	*	.*
하나의 모든 문자와 일치	?	.
characters 중 하나 이상에 포함되는 하나의 문자와 일치	[characters]	[characters]
characters에 포함되지 않는 하나의 문자와 일치	[!characters]	[^characters]



- 다양한 수의 표현 불가
 - 다룰 수 있는 수의 범위가 0-9
- 조건의 반복 불가
 - a-z까지의 문자가 세 번 반복되는 경우를 찾으려면 `[a-z][a-z][a-z]`와 같이 입력해야 함
- 느슨한 동작 불가
 - 어떠한 조건이 1번 이상, 3번 이하로 발생하는 경우를 정의할 수 없음



기호	뜻
<code>^</code>	문자열의 시작
<code>\$</code>	문자열의 끝
<code>[characters]</code>	characters에 해당하는 하나의 문자
<code>[^characters]</code>	characters에 해당하지 않는 하나의 문자
<code>.</code>	임의의 하나의 문자
<code>x y</code>	문자 x 혹은 y



기호	뜻
$x?$	0 혹은 하나의 x
x^*	0 혹은 여러 개의 x
x^+	1 혹은 여러 개의 x
$x\{y\}$	x 가 정확히 y 번 반복
$x\{y, \}$	x 가 y 번 이상 반복
$x\{y, z\}$	x 가 y 번 이상, z 번 이하로 반복
$(pattern)$	$pattern$ 을 하나로 묶음



- file0.txt, file1.txt, ...와 일치
 - `"^file[0-9]+\\.txt$"` → 시작: f(^), 끝: t(\$), file 이후 0-9 사이의 아무 문자([0-9])의 1 이상 반복(+)
- file1.txt, file2.txt, ..., file999.txt와 일치
 - `"^file[1-9][0-9]{0,2}\.txt$"` → file 이후 1-9와 일치(0 제외), 이후 0 이상 2 이하({0,2})의 0-9와 일치
- A1S, A2S, ..., A9S 혹은 B1S, B2S, ..., B9S와 일치
 - `"((A[1-9])|(B[1-9]))S"` → A와 1-9((A[1-9])) 혹은 B와 1-9((B[1-9])) 중 하나(|)와 일치하는 패턴(바깥 괄호) 이후의 S와 일치
- 항상 테스트하는 것이 좋음



- re 라이브러리를 이용하여 정규 표현식을 사용할 수 있음
- 정규 표현식 문자열은 따옴표 앞에 r을 붙여 표현(r''')
- **search**
 - 표현식과 문자열이 일치하는 가장 앞의 일치 객체 반환
- **fullmatch**
 - 표현식과 문자열이 완벽하게 일치하는지 확인하는 객체 반환
- **findall**
 - 표현식과 문자열이 일치하는 모든 일치 객체 반환
- **sub**
 - 문자열에서 표현식에 맞는 부분을 치환

```
import re

string = "파이의 근삿값은 3.14159265359입니다."
m = re.search(r"[1-9][0-9]*(\.[0-9]*)?", string)

if m:
    print(m.group())
    # 3.14159265359
```

- **search(표현식, 문자열)** 형태로 사용
- 처음으로 일치하는 문자열을 Match 객체로 반환
 - 만약 일치하는 문자열이 없으면 None을 반환
 - **Match.group** 메서드를 이용하여 일치 문자열 추출 가능


```
import re

string = "파이의 근삿값은 3.14159265359입니다."
m = re.fullmatch(r"[1-9][0-9]*(\.[0-9]*)?", string)

if m:
    print(m.group())
    # 아무 것도 출력하지 않음
```

- **fullmatch(표현식, 문자열)** 형태로 사용
- 표현식과 문자열 전체가 일치하는지 Match 객체로 반환
 - 만약 일치하는 문자열이 없으면 None을 반환
 - **search(표현식, "^문자열\$")**와 동일한 의미

```
import re

string = "사과는 영어로 apple, 바나나는 영어로 banana이다."
m = re.findall(r"[a-z]+", string)
for s in m:
    print(s)
    # apple
    # banana
```

- **findall(표현식, 문자열)** 형태로 사용
- 문자열 안에서 표현식과 일치하는 모든 부분 문자열을 리스트로 반환
 - 만약 일치하는 문자열이 없으면 빈 리스트를 반환

```
import re

string = "I like spam."
m = re.sub(r"[a-z]*", "hate", string)

print(m)
# I hate spam.
```

- **sub(표현식, 치환 문자열, 문자열)** 형태로 사용
- 문자열에서 표현식과 일치하는 요소를 치환 문자열로 대체하여 문자열로 반환
 - 일치하는 문자열이 없으면 원 문자열 반환(치환되지 않음)



정규 표현식을 이용한 파일 찾기 예시

Python을 이용한 파일
처리

03 여러 파일 다루기

2. 간단한 정규 표현식

```
import re
from pathlib import Path

def find_matching_files(directory, pattern):
    """정규 표현식에 해당하는 파일을 디렉터리에서 찾아 출력"""
    p = Path(directory)

    # 디렉터리 순회
    for file in p.iterdir():
        # 디렉터리가 아닌지 확인
        if file.is_file():
            # 정규 표현식으로 부분이 아닌 전체 확인
            m = re.fullmatch(pattern, file.name)

            if m:
                print(m.group())

find_matching_files("./", r"file[0-9]+\\.txt")
# 작업 디렉터리에서 file1.txt, file2.txt, ...를 모두 찾아 출력
```